Equipotential Visualizer

Generated by Doxygen 1.9.8

1 Equipotential Simulator/Visualizer 1
1.1 Features . . . 1
1.2 Demonstrated Skills . . . 1
1.3 Installation & Build . . . 1
1.4 Compile (from src directory or adjust paths) . . . 1
1.4.1 Usage Example . . . 2
1.4.2 Roadmap . . . 2
1.4.3 License . . . 2

2 Data Structure Index 3
2.1 Data Structures . . . 3

3 File Index 5
3.1 File List . . . 5

4 Data Structure Documentation 7
4.1 charge_t Struct Reference . . . 7
4.1.1 Detailed Description . . . 7
4.1.2 Field Documentation . . . 7
4.1.2.1 magnitude . . . 7
4.1.2.2 point_x . . . 7
4.1.2.3 point_y . . . 8
4.1.2.4 sign . . . 8
4.1.2.5 space_x . . . 8
4.1.2.6 space_y . . . 8

5 File Documentation 9
5.1 README.md File Reference . . . 9
5.2 src/charge.h File Reference . . . 9
5.2.1 Typedef Documentation . . . 10
5.2.1.1 charge_t . . . 10
5.2.2 Enumeration Type Documentation . . . 10
5.2.2.1 sign_t . . . 10
5.3 charge.h . . . 10
5.4 src/config.h File Reference . . . 11
5.4.1 Macro Definition Documentation . . . 12
5.4.1.1 CONTOUR_DECADES . . . 12
5.4.1.2 CONTOUR_LINES . . . 12
5.4.1.3 COULOMBS_CONSTANT . . . 12
5.4.1.4 NX . . . 12
5.4.1.5 NY . . . 12
5.4.1.6 VOLT_INC . . . 12
5.4.1.7 XMAX . . . 12
5.4.1.8 XMIN . . . 13

5.4.1.9 YMAX . . . 13
5.4.1.10 YMIN . . . 13
5.5 config.h . . . 13
5.6 src/grid.h File Reference . . . 14
5.6.1 Variable Documentation . . . 14
5.6.1.1 dx . . . 14
5.6.1.2 dy . . . 15
5.7 grid.h . . . 15
5.8 src/main.c File Reference . . . 15
5.8.1 Function Documentation . . . 16
5.8.1.1 main() . . . 16
5.8.2 Variable Documentation . . . 16
5.8.2.1 dx . . . 16
5.8.2.2 dy . . . 16
5.8.2.3 V . . . 16
5.9 main.c . . . 17
5.10 src/render.c File Reference . . . 18
5.10.1 Macro Definition Documentation . . . 18
5.10.1.1 STB_IMAGE_WRITE_IMPLEMENTATION . . . 18
5.10.2 Function Documentation . . . 19
5.10.2.1 draw_contours_for_sign() . . . 19
5.10.2.2 find_voltage_extrema() . . . 19
5.10.2.3 render_voltage_based() . . . 20
5.11 render.c . . . 21
5.12 src/render.h File Reference . . . 23
5.12.1 Function Documentation . . . 24
5.12.1.1 render_voltage_based() . . . 24
5.13 render.h . . . 25
5.14 src/space_calc.c File Reference . . . 25
5.14.1 Function Documentation . . . 26
5.14.1.1 calc_volt() . . . 26
5.14.1.2 setup_grid() . . . 27
5.15 space_calc.c . . . 27
5.16 src/space_calc.h File Reference . . . 28
5.16.1 Function Documentation . . . 29
5.16.1.1 setup_grid() . . . 29
5.17 space_calc.h . . . 30

Index **31**

Chapter 1

Equipotential Simulator/Visualizer

A high-resolution 2D electrostatic equipotential line simulator written in C.

1.1 Features

- User defined point charges
- High resolution rendering and output (2000x1000 by default)
- Contour drawing using LOG based mathematics
- Written in pure C (only external dependency being STB)

1.2 Demonstrated Skills

- **Low-level programming and mathematics**: This program demonstrates low level mathematics and memory management.
- **C Proficiency**: The lack of external dependecies and pure C showcases a proficiency in a highly-tedious and unforgiving programming language.
  - Modular design with header/source separation, structs, enums.
  - Grid-based field computation, distance calculations, avoiding singularities.
- **Image processing**: Integration with `stb_image_write.h` for PNG output to desired location.

1.3 Installation & Build

```bash git clone https://github.com/Shrewd-Man/Equipotential-Visualizer.git cd Equipotential-Visualizer

## 1.4 Compile (from src directory or adjust paths)

gcc src/*.c -o equipotential -lm -I src ```Run./equipotential`
```

1.4.1 Usage Example

- 2 charges: +1 nC at (-2, 0), -1 nC at (2, 0) $\rightarrow$ classic dipole equipotential pattern.
- Supports up to 4 charges currently (easily extensible).

1.4.2 Roadmap

- **First Change**: addition of a grayscale heatmap-like render to outperform flat contours.
- Modular input to allow implementation in custom applications.
- Real-time data output formatted for custom rendering programs.
- 3D extention.

1.4.3 License

Use in compliance with the GPL-3.0 license

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

charge_t 7

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

src/charge.h 9
src/config.h 11
src/grid.h 14
src/main.c 15
src/render.c 18
src/render.h 23
src/space_calc.c 25
src/space_calc.h 28

Chapter 4

Data Structure Documentation

4.1 charge_t Struct Reference

`#include <charge.h>`

Data Fields

- double space_x
- double space_y
- int point_x
- int point_y
- double magnitude
- sign_t sign

4.1.1 Detailed Description

Definition at line 13 of file charge.h.

4.1.2 Field Documentation

4.1.2.1 magnitude

`double charge_t::magnitude`

Definition at line 20 of file charge.h.

4.1.2.2 point_x

`int charge_t::point_x`

Definition at line 17 of file charge.h.

4.1.2.3 point_y

```
int charge_t::point_y
```

Definition at line 18 of file charge.h.

4.1.2.4 sign

```
sign_t charge_t::sign
```

Definition at line 22 of file charge.h.

4.1.2.5 space_x

```
double charge_t::space_x
```

Definition at line 14 of file charge.h.

4.1.2.6 space_y

```
double charge_t::space_y
```

Definition at line 15 of file charge.h.

The documentation for this struct was generated from the following file:

- src/charge.h

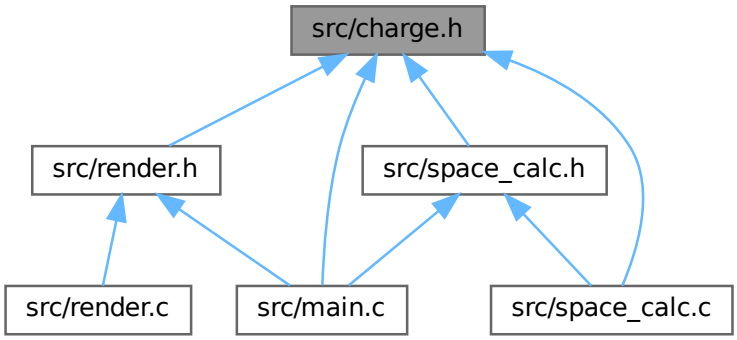
Chapter 5

File Documentation

5.1 README.md File Reference

5.2 src/charge.h File Reference

This graph shows which files directly or indirectly include this file:



Data Structures

- struct charge_t

Typedefs

- typedef struct charge_t charge_t

Enumerations

- enum sign_t { NEGATIVE = 0 , POSITIVE = 1 }

5.2.1 Typedef Documentation

5.2.1.1 charge_t

```
typedef struct charge_t charge_t
```

5.2.2 Enumeration Type Documentation

5.2.2.1 sign_t

```
enum sign_t
```

Enumerator

NEGATIVE	
POSITIVE	

Definition at line 7 of file charge.h.

5.3 charge.h

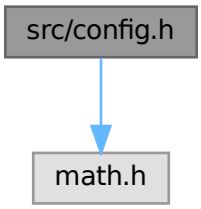
Go to the documentation of this file.

```
00001 // charge.h
00002
00003 #ifndef CHARGE_H
00004 #define CHARGE_H
00005
00006 // --- Sign Bit ---
00007 typedef enum {
00008     NEGATIVE = 0,
00009     POSITIVE = 1
00010 } sign_t;
00011
00012 // --- Charge Structure ---
00013 typedef struct charge_t {
00014     double space_x;  /* Meters */
00015     double space_y;  /* Meters */
00016
00017     int point_x; /* Point-based Coordinates */
00018     int point_y; /* Point-based Coordinates */
00019
00020     double magnitude; /* Coulombs */
00021
00022     sign_t sign; /* 0 for negative, 1 for positive */
00023 } charge_t;
00024
00025 #endif //CHARGE_H
```

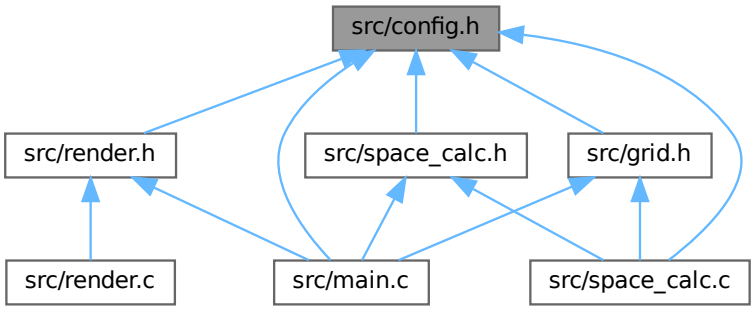
5.4 src/config.h File Reference

```
#include <math.h>
```

Include dependency graph for config.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define XMIN -10.0 /∗ Meters ∗/
- #define XMAX 10.0 /∗ Meters ∗/
- #define YMIN -5.0 /∗ Meters ∗/
- #define YMAX 5.0 /∗ Meters ∗/
- #define NX 2000
- #define NY 1000
- #define VOLT_INC 15
- #define CONTOUR_LINES 8 /∗ Contour lines drawn for each charge sign ∗/
- #define CONTOUR_DECADES 2.0 /∗ How far below its peak a sign's contours reach, in powers of ten ∗/
- #define COULOMBS_CONSTANT (8.99e9)

5.4.1 Macro Definition Documentation

5.4.1.1 CONTOUR_DECADES

```
#define CONTOUR_DECADES 2.0 /* How far below its peak a sign's contours reach, in powers of
ten */
```

Definition at line 25 of file config.h.

5.4.1.2 CONTOUR_LINES

```
#define CONTOUR_LINES 8 /* Contour lines drawn for each charge sign */
```

Definition at line 24 of file config.h.

5.4.1.3 COULOMBS_CONSTANT

```
#define COULOMBS_CONSTANT (8.99e9)
```

Definition at line 30 of file config.h.

5.4.1.4 NX

```
#define NX 2000
```

Definition at line 17 of file config.h.

5.4.1.5 NY

```
#define NY 1000
```

Definition at line 18 of file config.h.

5.4.1.6 VOLT_INC

```
#define VOLT_INC 15
```

Definition at line 21 of file config.h.

5.4.1.7 XMAX

```
#define XMAX 10.0 /* Meters */
```

Definition at line 12 of file config.h.

5.4.1.8 XMIN

```
#define XMIN -10.0 /* Meters */
```

Definition at line 11 of file config.h.

5.4.1.9 YMAX

```
#define YMAX 5.0 /* Meters */
```

Definition at line 14 of file config.h.

5.4.1.10 YMIN

```
#define YMIN -5.0 /* Meters */
```

Definition at line 13 of file config.h.

5.5 config.h

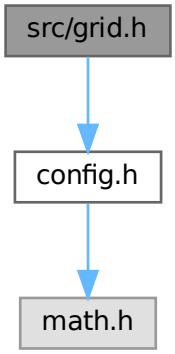
Go to the documentation of this file.

```
00001 // CONFIG.H
00002 #ifndef CONFIG_H
00003 #define CONFIG_H
00004
00005 // --- NECESSARY HEADERS ---
00006 #include <math.h>
00007
00008 // --- SIMULATION PARAMETERS ---
00009
00010     // Boundaries
00011 #define XMIN -10.0 /* Meters */
00012 #define XMAX 10.0  /* Meters */
00013 #define YMIN -5.0  /* Meters */
00014 #define YMAX 5.0   /* Meters */
00015
00016     // Resolution
00017 #define NX 2000
00018 #define NY 1000
00019
00020     // Voltage-Based Rendering increments
00021 #define VOLT_INC 15
00022
00023     // Voltage-Based Rendering contours
00024 #define CONTOUR_LINES 8      /* Contour lines drawn for each charge sign */
00025 #define CONTOUR_DECADES 2.0  /* How far below its peak a sign's contours reach, in powers of ten */
00026
00027 // --- CONSTANTS ---
00028
00029     // Coulombs Constant
00030 #define COULOMBS_CONSTANT (8.99e9)
00031
00032
00033 #endif
```

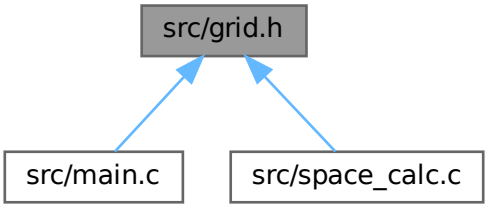
5.6 src/grid.h File Reference

```
#include "config.h"
```

Include dependency graph for grid.h:



This graph shows which files directly or indirectly include this file:



Variables

- double dx
- double dy

5.6.1 Variable Documentation

5.6.1.1 dx

```
double dx  [extern]
```

Definition at line 18 of file main.c.

5.6.1.2 dy

```
double dy  [extern]
```

Definition at line 19 of file main.c.

5.7 grid.h

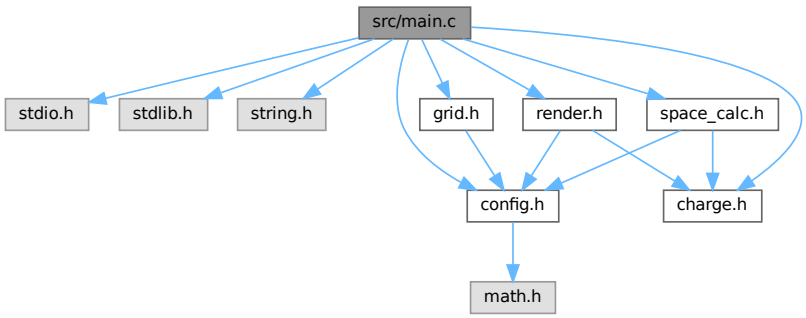
Go to the documentation of this file.

```
00001 // grid.h
00002 #ifndef GRID_H
00003 #define GRID_H
00004
00005 #include "config.h"
00006
00007 // --- UNIVERSAL GRID VALUES ---
00008 extern double dx;
00009 extern double dy;
00010
00011 #endif
```

5.8 src/main.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "config.h"
#include "charge.h"
#include "render.h"
#include "grid.h"
#include "space_calc.h"
```

Include dependency graph for main.c:



Functions

- int main (int argc, const char *argv[])

Variables

- double V [NX][NY]
- double dx = (XMAX - XMIN) / (NX - 1)
- double dy = (YMAX - YMIN) / (NY - 1)

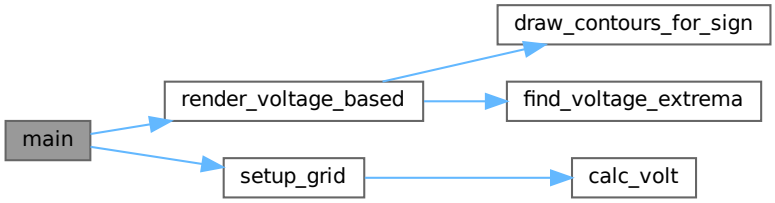
5.8.1 Function Documentation

5.8.1.1 main()

```
int main (
            int argc,
            const char * argv[] )
```

Definition at line 21 of file main.c.

Here is the call graph for this function:



5.8.2 Variable Documentation

5.8.2.1 dx

```
double dx = (XMAX - XMIN) / (NX - 1)
```

Definition at line 18 of file main.c.

5.8.2.2 dy

```
double dy = (YMAX - YMIN) / (NY - 1)
```

Definition at line 19 of file main.c.

5.8.2.3 V

```
double V[NX][NY]
```

Definition at line 17 of file main.c.

5.9 main.c

Go to the documentation of this file.

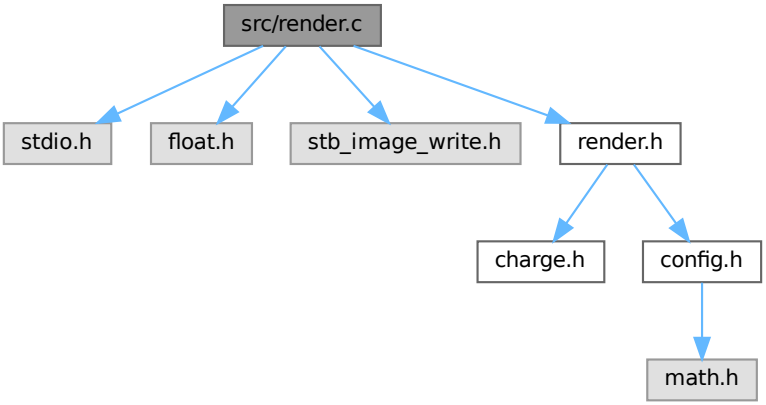
```
00001 //
00002 //  main.c
00003 //  Equipotential-Visualizer
00004 //
00005 //  Created by Kenneth Anderson on 2/15/25.
00006 //
00007
00008 #include <stdio.h>
00009 #include <stdlib.h>
00010 #include <string.h>
00011 #include "config.h"
00012 #include "charge.h"
00013 #include "render.h"
00014 #include "grid.h"
00015 #include "space_calc.h"
00016
00017 double V[NX][NY];
00018 double dx = (XMAX - XMIN) / (NX - 1);
00019 double dy = (YMAX - YMIN) / (NY - 1);
00020
00021 int main(int argc, const char * argv[]) {
00022
00023     // create array of 4 charges
00024     charge_t charge[4];
00025
00026     // Create an int to define the amount of charges the user has requested
00027     int charge_count;
00028
00029     // create input buffers (charge size entry, and the output image path)
00030     char str_in[32];
00031     char output_path[256];
00032
00033     printf("Enter the amount of charges: ");
00034     scanf("%d", &charge_count);
00035
00036     if(charge_count < 5 && charge_count > 0) {
00037         for(int i = 0; i < charge_count; i++) {
00038             // display charge number and request values
00039             printf("Charge %d:\nEnter the x position of the charge: ", i + 1);
00040             scanf("%lf", &charge[i].space_x);
00041             printf("Enter the y position of charge: ");
00042             scanf("%lf", &charge[i].space_y);
00043             printf("Enter the charge size (enter x to default to one positive nanocoulomb and z to
      default to one negative nanocoulomb): ");
00044             scanf("%31s", str_in); // width limit leaves room for the null terminator
00045
00046             // compare string and assign auto-values as needed
00047             if(strcmp(str_in, "x") == 0) {
00048                 charge[i].magnitude = (1.0e-9); // 1 nanocoulomb
00049                 charge[i].sign = POSITIVE;
00050             } else if (strcmp(str_in, "z") == 0) {
00051                 charge[i].magnitude = (1.0e-9); // 1 nanocoulomb
00052                 charge[i].sign = NEGATIVE;
00053             } else {
00054                 // custom value in coulombs: the sign comes from the value, magnitude is stored as
      absolute
00055                 double value = atof(str_in);
00056                 charge[i].sign = (value < 0) ? NEGATIVE : POSITIVE;
00057                 charge[i].magnitude = fabs(value);
00058             }
00059
00060             // assign and set the point coordinates of the charges based on the spatial coordinates
00061             charge[i].point_x = (int)(((charge[i].space_x - XMIN) / dx) + 0.5);
00062             charge[i].point_y = (int)(((charge[i].space_y - YMIN) / dy) + 0.5);
00063         }
00064     } else {
00065         printf("Inputted value exceeds maximum. Terminating session.");
00066         return 0;
00067     }
00068
00069     // calculate all ΔV values
00070     printf("Setting up...\n");
00071     setup_grid(charge, charge_count, V);
00072     printf("Setup Complete.\n\n");
00073
00074     printf("\nVoltage calculation successful. Please enter the filepath for the rendered image
      (include filename and '.png' extension): ");
00075     scanf("%255s", output_path);
00076
00077     // render the image
00078     printf("Rendering...\n");
00079     render_voltage_based(charge, charge_count, V, output_path);
```

```
00080     printf("Rendering Complete.\n\n");
00081
00082     // exit the program
00083     return 0;
00084 }
```

5.10 src/render.c File Reference

```
#include <stdio.h>
#include <float.h>
#include "stb_image_write.h"
#include "render.h"
```

Include dependency graph for render.c:



Macros

- #define STB_IMAGE_WRITE_IMPLEMENTATION

Functions

- double find_voltage_extrema (charge_t charge[ ], int charge_count, int type, double V[NX][NY])

  function finds the minimum and maximum voltage in the space, and returns either min or max
- void draw_contours_for_sign (double V[NX][NY], unsigned char img[NY][NX], int sign, double extremum, int num_lines)

  draws the equipotential contour lines for either the positive or the negative potentials
- void render_voltage_based (charge_t charge[ ], int charge_count, double V[NX][NY], const char ∗output_↩ path)

  This function approaches the rendering by using the voltage of the space.

5.10.1 Macro Definition Documentation

5.10.1.1 STB_IMAGE_WRITE_IMPLEMENTATION

```
#define STB_IMAGE_WRITE_IMPLEMENTATION
```

Definition at line 5 of file render.c.

5.10.2 Function Documentation

5.10.2.1 draw_contours_for_sign()

```
void draw_contours_for_sign (
            double V[NX][NY],
            unsigned char img[NY][NX],
            int sign,
            double extremum,
            int num_lines )
```

draws the equipotential contour lines for either the positive or the negative potentials

The contour levels are spaced logarithmically between the weakest and strongest potential of the requested sign (never more than CONTOUR_DECADES powers of ten below the strongest, so the lines stay in the part of the field that actually has structure).

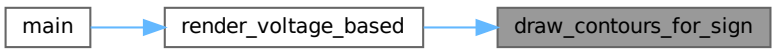
Every point is assigned the number of contour levels it has passed, and a line is drawn wherever two neighbouring points have a different count. This keeps every line a constant, thin width no matter how steep or shallow the potential is, instead of painting a band around each level.

Parameters

V	the voltage coordinate array
img	the image to draw the lines into (255 for a line)
sign	+1 to draw the contours of the positive potentials, -1 for the negative potentials
extremum	the strongest voltage of this sign (taken outside the charges' exclusion zone)
num_lines	the number of contour lines to draw for this sign (2 or more)

Definition at line 64 of file render.c.

Here is the caller graph for this function:



5.10.2.2 find_voltage_extrema()

```
double find_voltage_extrema (
            charge_t charge[],
            int charge_count,
            int type,
            double V[NX][NY] )
```

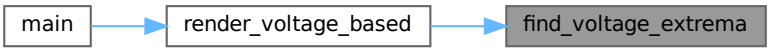
function finds the minimum and maximum voltage in the space, and returns either min or max

Parameters

type	indicates return; 1 for max, 0 for min
V	array for voltage calcs

Definition at line 18 of file render.c.

Here is the caller graph for this function:



5.10.2.3 render_voltage_based()

```
void render_voltage_based (
            charge_t charge[],
            int charge_count,
            double V[NX][NY],
            const char * output_path )
```

This function approaches the rendering by using the voltage of the space.

The function performs as follows:

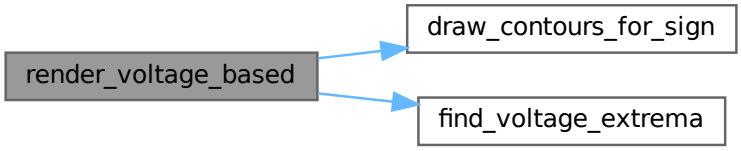
- The function first determines the maximum and minimum voltage in the space
- Then, for the positive and the negative voltages separately, CONTOUR_LINES log-spaced voltage levels are chosen
- A line is drawn (white) wherever two neighbouring points (i, j) fall between different levels, everything else stays black

Parameters

charge	the structure containing the data for the first charge
charge_count	the total amounts of charges
V	the voltage coordinate array for which to render
output_path	the filepath (including the '.png' extension) to write the rendered image to

Definition at line 117 of file render.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.11 render.c

Go to the documentation of this file.

```
00001 // render.c
00002 #include <stdio.h>
00003 #include <float.h>
00004
00005 #define STB_IMAGE_WRITE_IMPLEMENTATION
00006 #include "stb_image_write.h"
00007
00008 #include "render.h"
00009
00010
00011 // --- Voltage Extrema Function ---
00018 double find_voltage_extrema(charge_t charge[], int charge_count, int type, double V[NX][NY]) {
00019     // start from the opposite ends so a charge sitting on the V[0][0] corner can't poison the result
00020     double v_min = DBL_MAX, v_max = -DBL_MAX;
00021     double distance_to_charge;
00022     int too_close = 0;
00023     // use a for loop to calculate values
00024     for (int i = 0; i < NX; i++) {
00025         for (int j = 0; j < NY; j++) {
00026             too_close = 0;
00027
00028             for (int k = 0; k < charge_count; k++) {
00029                 distance_to_charge = sqrt(pow((i - charge[k].point_x), 2) + pow((j -
      charge[k].point_y), 2));
00030                 if (distance_to_charge < 30) {
00031                     too_close = 1;
00032                 }
00033             }
00034             if (too_close == 0) {
00035                 if (V[i][j] < v_min) v_min = V[i][j];
00036                 if (V[i][j] > v_max) v_max = V[i][j];
00037             }
00038             //if (V[i][j] < v_min) v_min = V[i][j];
00039             //if (V[i][j] > v_max) v_max = V[i][j];
00040             //printf("%.2f is min\n %.2f is max\n\n", v_min, v_max);
00041         }
00042     }
```

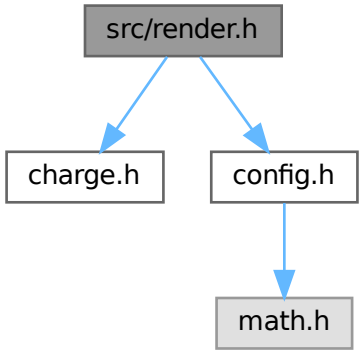
```
00043     return type ? v_max : v_min; // return value based on type
00044 }
00045
00046 // --- Contour Drawing Function ---
00064 void draw_contours_for_sign(double V[NX][NY], unsigned char img[NY][NX], int sign, double extremum,
      int num_lines) {
00065     double v_hi = fabs(extremum);
00066     if (v_hi < 1e-12 || num_lines < 2 || num_lines > 250) return;
00067
00068     // find the weakest potential of this sign that exists anywhere in the field
00069     double v_seen_min = v_hi;
00070     for (int i = 0; i < NX; i++) {
00071         for (int j = 0; j < NY; j++) {
00072             double mag = sign * V[i][j]; // positive when the point has the requested sign
00073             if (mag > 0 && mag < v_seen_min) v_seen_min = mag;
00074         }
00075     }
00076
00077     // the dipole's zero line makes the weakest potential tiny, so limit how far down the levels go
00078     double v_lo = fmax(v_seen_min, fmax(v_hi * pow(10.0, -CONTOUR_DECADES), 1e-12));
00079     if (v_lo >= v_hi) return;
00080
00081     // place the levels strictly inside [v_lo, v_hi] so that every one of them is visible
00082     double log_lo = log10(v_lo);
00083     double step = (log10(v_hi) - log_lo) / (num_lines + 1);
00084
00085     // band[i][j] is how many levels the point has passed (0 = below the first level)
00086     static unsigned char band[NX][NY];
00087     for (int i = 0; i < NX; i++) {
00088         for (int j = 0; j < NY; j++) {
00089             double mag = sign * V[i][j];
00090             int b = 0;
00091             if (mag > 0) {
00092                 b = (int)floor((log10(mag) - log_lo) / step);
00093                 if (b < 0) b = 0;
00094                 if (b > num_lines) b = num_lines;
00095             }
00096             band[i][j] = (unsigned char)b;
00097         }
00098     }
00099
00100     // draw a line wherever neighbouring points fall in different bands
00101     for (int i = 0; i < NX; i++) {
00102         for (int j = 0; j < NY; j++) {
00103             if (i + 1 < NX && band[i][j] != band[i + 1][j]) {
00104                 img[j][i] = 255;
00105                 img[j][i + 1] = 255;
00106             }
00107             if (j + 1 < NY && band[i][j] != band[i][j + 1]) {
00108                 img[j][i] = 255;
00109                 img[j + 1][i] = 255;
00110             }
00111         }
00112     }
00113 }
00114
00115 // --- Rendering Function ---
00116
00117 void render_voltage_based(charge_t charge[], int charge_count, double V[NX][NY], const char
      *output_path) {
00118     printf("Starting beginning extrema sequence...\n");
00119
00120     // Find global voltage extrema across the field (ignores the points right next to the charges)
00121     double v_min = find_voltage_extrema(charge, charge_count, 0, V);
00122     double v_max = find_voltage_extrema(charge, charge_count, 1, V);
00123
00124     printf("Sequence succeeded. Rendering %d voltage lines per charge sign...\n", CONTOUR_LINES);
00125
00126     // Create the image
00127     unsigned char img[NY][NX];
00128     for (int i = 0; i < NX; i++) {
00129         for (int j = 0; j < NY; j++) {
00130             img[j][i] = 0;  // Black background
00131         }
00132     }
00133
00134     // Draw positive contours
00135     if (v_max > 1e-12) {
00136         draw_contours_for_sign(V, img, +1, v_max, CONTOUR_LINES);
00137     }
00138
00139     // Draw negative contours
00140     if (v_min < -1e-12) {
00141         draw_contours_for_sign(V, img, -1, v_min, CONTOUR_LINES);
00142     }
00143
00144     // Set the charge position to a white dot
```

```
00145     for (int i = 0; i < charge_count; i++) {
00146         int cx = charge[i].point_x;
00147         int cy = charge[i].point_y;
00148         img[cy][cx] = 255;  // Mark each charge
00149     }
00150
00151     // Write the image to disk
00152     stbi_write_png(output_path, NX, NY, 1, img, NX);
00153 }
```

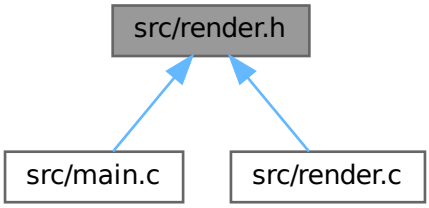
5.12 src/render.h File Reference

```
#include "charge.h"
#include "config.h"
```

Include dependency graph for render.h:



This graph shows which files directly or indirectly include this file:



Functions

- void render_voltage_based (charge_t charge[ ], int charge_count, double V[NX][NY], const char ∗output_↩ path)

  This function approaches the rendering by using the voltage of the space.

5.12.1 Function Documentation

5.12.1.1 render_voltage_based()

```
void render_voltage_based (
            charge_t charge[],
            int charge_count,
            double V[NX][NY],
            const char * output_path )
```

This function approaches the rendering by using the voltage of the space.

The function performs as follows:

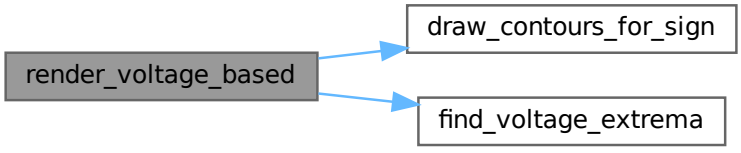
- The function first determines the maximum and minimum voltage in the space
- Then, for the positive and the negative voltages separately, CONTOUR_LINES log-spaced voltage levels are chosen
- A line is drawn (white) wherever two neighbouring points (i, j) fall between different levels, everything else stays black

Parameters

charge	the structure containing the data for the first charge
charge_count	the total amounts of charges
V	the voltage coordinate array for which to render
output_path	the filepath (including the '.png' extension) to write the rendered image to

Definition at line 117 of file render.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.13 render.h

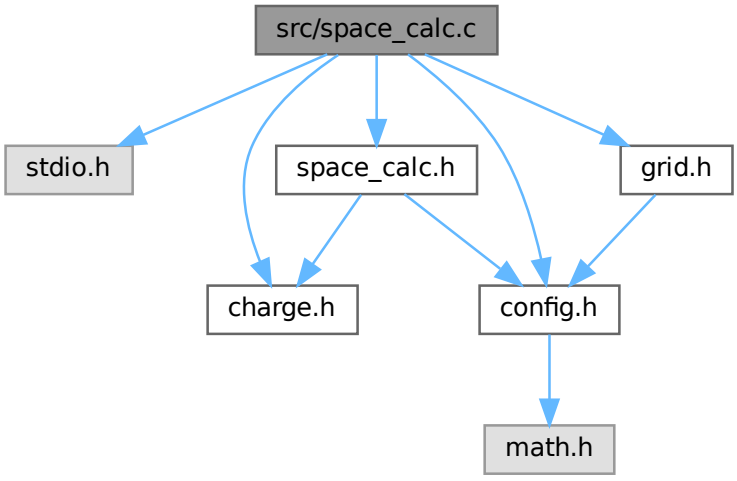
Go to the documentation of this file.

```
00001 // render.h
00002 #ifndef RENDER_H
00003 #define RENDER_H
00004
00005 #include "charge.h"
00006 #include "config.h"
00007
00008 // --- VOLTAGE BASED RENDERING ---
00009
00023 void render_voltage_based(charge_t charge[], int charge_count, double V[NX][NY], const char
      *output_path);
00024
00025 #endif
```

5.14 src/space_calc.c File Reference

```
#include <stdio.h>
#include "space_calc.h"
#include "charge.h"
#include "config.h"
#include "grid.h"
```

Include dependency graph for space_calc.c:



Functions

- double calc_volt (charge_t charge[ ], int charge_count, int x_grid, int y_grid)

  Function calculates the potential difference of any provided point.
- void setup_grid (charge_t charge[ ], int charge_count, double V[NX][NY])

  Function calculates the potential difference of every point in the matrix.

5.14.1 Function Documentation

5.14.1.1 calc_volt()

```
double calc_volt (
            charge_t charge[],
            int charge_count,
            int x_grid,
            int y_grid )
```

Function calculates the potential difference of any provided point.

The function uses the universal formula and currently computes that based on the points distance to the charge x and y

Parameters

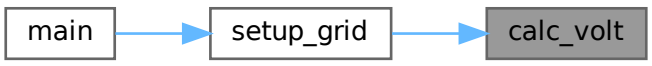
charge	structure data with the charge values
charge_count	total amount of charges
x_grid	The x grid point of the calculation
y_grid	The y grid point of the calculation

Returns

Func returns the value of the potential difference at the given point

Definition at line 20 of file space_calc.c.

Here is the caller graph for this function:



5.14.1.2 setup_grid()

```
void setup_grid (
            charge_t charge[],
            int charge_count,
            double V[NX][NY] )
```

Function calculates the potential difference of every point in the matrix.

The function calls the calc_volt for every point in the space

Parameters

charge	the structure containing the data for the first charge
charge_count	the total amounts of charges

Note

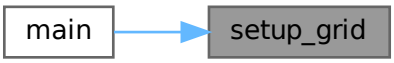
This function is currently a temporary system that does not allow for live rendering

Definition at line 43 of file space_calc.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.15 space_calc.c

Go to the documentation of this file.

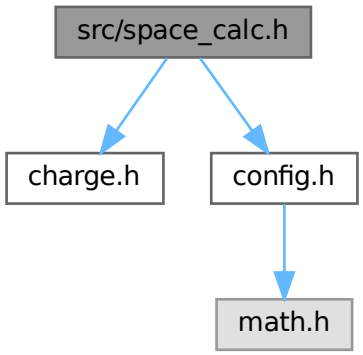
```
00001 #include <stdio.h>
00002 #include "space_calc.h"
00003 #include "charge.h"
```

```
00004 #include "config.h"
00005 #include "grid.h"
00006
00007 // --- Single Point Voltage Calculation Algorithm ---
00020 double calc_volt(charge_t charge[], int charge_count, int x_grid, int y_grid) {
00021     double x_phys = XMIN + x_grid * dx;
00022     double y_phys = YMIN + y_grid * dy;
00023     double potential = 0.0;
00024     for (int i = 0; i < charge_count; i++) {
00025         double distance_to_charge = sqrt(pow(x_phys - charge[i].space_x, 2) + pow(y_phys -
      charge[i].space_y, 2));
00026         double q = (charge[i].sign == POSITIVE) ? charge[i].magnitude : -charge[i].magnitude;
00027         potential += (COULOMBS_CONSTANT * q) / (distance_to_charge + 1e-9);
00028         //potential += (COULOMBS_CONSTANT * charge[i].magnitude) / (distance_to_charge + 1e-9);
00029     }
00030     return potential;
00031 }
00032
00033 // --- Loop System for Each Point ---
00043 void setup_grid(charge_t charge[], int charge_count, double V[NX][NY]) {
00044     for (int i = 0; i < NX; i++) {
00045         for (int j = 0; j < NY; j++) {
00046             V[i][j] = calc_volt(charge, charge_count, i, j); // Use to calculate the difference
00047             if (i % 500 == 0 && j % 250 == 0) {  // Sample every 500x250 points
00048                 printf("V[%d][%d] = %.2e\n", i, j, V[i][j]);
00049             }
00050         }
00051     }
00052 }
```

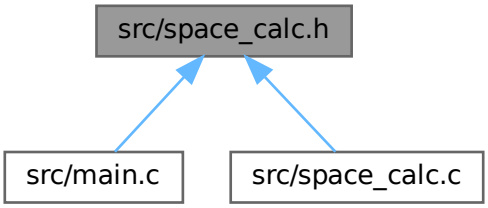
5.16 src/space_calc.h File Reference

```
#include "charge.h"
#include "config.h"
```

Include dependency graph for space_calc.h:



This graph shows which files directly or indirectly include this file:



Functions

- void setup_grid (charge_t charge[ ], int charge_count, double V[NX][NY])

  Function calculates the potential difference of every point in the matrix.

5.16.1 Function Documentation

5.16.1.1 setup_grid()

```
void setup_grid (
            charge_t charge[],
            int charge_count,
            double V[NX][NY] )
```

Function calculates the potential difference of every point in the matrix.

The function calls the calc_volt for every point in the space

Parameters

charge	the structure containing the data for the first charge
charge_count	the total amounts of charges

Note

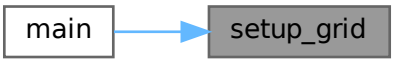
This function is currently a temporary system that does not allow for live rendering

Definition at line 43 of file space_calc.c.

Here is the call graph for this function:



Here is the caller graph for this function:



5.17 space_calc.h

Go to the documentation of this file.

```
00001 // space_calc.h
00002 #ifndef SPACE_CALC_H
00003 #define SPACE_CALC_H
00004
00005 #include "charge.h"
00006 #include "config.h"
00007
00008 // --- VOLTAGE SPACE CALCULATION ALGORITHM ---
00009
00019 void setup_grid(charge_t charge[], int charge_count, double V[NX][NY]);
00020
00021
00022 #endif
```

Index

calc_volt
  space_calc.c, 26
charge.h
  charge_t, 10
  NEGATIVE, 10
  POSITIVE, 10
  sign_t, 10
charge_t, 7
  charge.h, 10
  magnitude, 7
  point_x, 7
  point_y, 7
  sign, 8
  space_x, 8
  space_y, 8
config.h
  CONTOUR_DECADES, 12
  CONTOUR_LINES, 12
  COULOMBS_CONSTANT, 12
  NX, 12
  NY, 12
  VOLT_INC, 12
  XMAX, 12
  XMIN, 12
  YMAX, 13
  YMIN, 13
CONTOUR_DECADES
  config.h, 12
CONTOUR_LINES
  config.h, 12
COULOMBS_CONSTANT
  config.h, 12

draw_contours_for_sign
  render.c, 19
dx
  grid.h, 14
  main.c, 16
dy
  grid.h, 14
  main.c, 16

Equipotential Simulator/Visualizer, 1

find_voltage_extrema
  render.c, 19

grid.h
  dx, 14
  dy, 14

magnitude
  charge_t, 7
main
  main.c, 16
main.c
  dx, 16
  dy, 16
  main, 16
  V, 16

NEGATIVE
  charge.h, 10
NX
  config.h, 12
NY
  config.h, 12

point_x
  charge_t, 7
point_y
  charge_t, 7
POSITIVE
  charge.h, 10

README.md, 9
render.c
  draw_contours_for_sign, 19
  find_voltage_extrema, 19
  render_voltage_based, 20
  STB_IMAGE_WRITE_IMPLEMENTATION, 18
render.h
  render_voltage_based, 24
render_voltage_based
  render.c, 20
  render.h, 24

setup_grid
  space_calc.c, 26
  space_calc.h, 29
sign
  charge_t, 8
sign_t
  charge.h, 10
space_calc.c
  calc_volt, 26
  setup_grid, 26
space_calc.h
  setup_grid, 29
space_x
  charge_t, 8

space_y
    charge_t, 8
src/charge.h, 9, 10
src/config.h, 11, 13
src/grid.h, 14, 15
src/main.c, 15, 17
src/render.c, 18, 21
src/render.h, 23, 25
src/space_calc.c, 25, 27
src/space_calc.h, 28, 30
STB_IMAGE_WRITE_IMPLEMENTATION
    render.c, 18

V
    main.c, 16
VOLT_INC
    config.h, 12

XMAX
    config.h, 12
XMIN
    config.h, 12

YMAX
    config.h, 13
YMIN
    config.h, 13